



AWS IoT Core - Publish and Receive Messages on Linux

Submitted to: Dr Abdelrhman Abotaleb & Dr Ahmed Hussein

Done by: Mohamed Ahmed Abdelazim Mohamed

Introduction

This document outlines the steps required to create a C++ application that can publish and receive messages using AWS IoT Core service.

Prerequisites

1. AWS IoT Core Setup:

- Navigate to the AWS IoT Core console.
- Create a new IoT Thing, generate security certificates, and download the following files:
 - Device Certificate (.crt)
 - Private Key (.key)
 - Amazon Root CA1 certificate (.pem)
- Get **AWS IoT endpoint** (format: xxxxxxxxxxxxxxxx-ats.iot.<region>.amazonaws.com).

Steps in the console

1- Click on Things here:

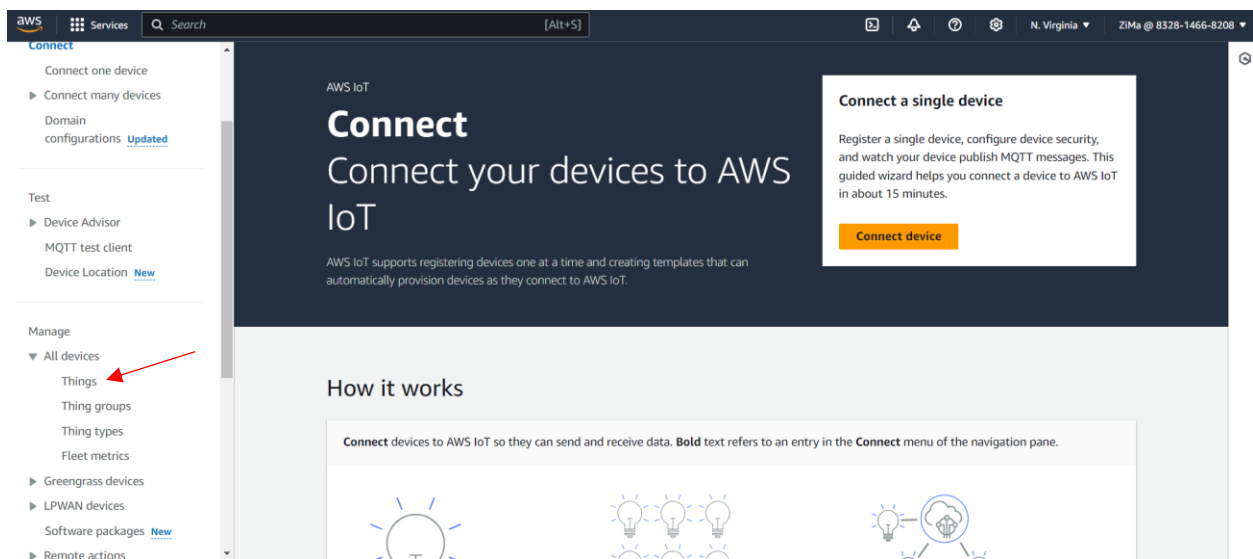


Figure 1 step 1 in creating thing

2- Click on Create things

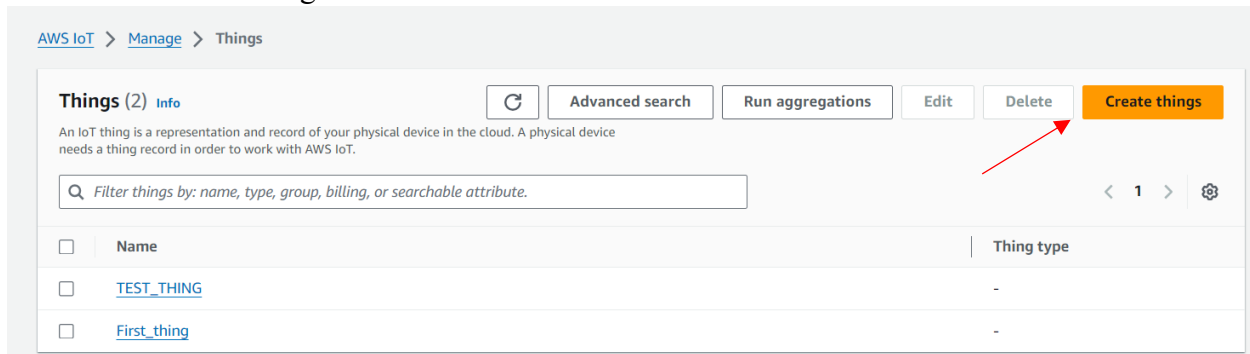


Figure 2 step 1 in creating thing

3- name the thing and then click on next leaving every option as the default.

Thing name

Enter a unique name containing only: letters, numbers, hyphens, colons, or underscores. A thing name can't contain any spaces.

Additional configurations

You can use these configurations to add detail that can help you to organize, manage, and search your things.

- ▶ Thing type - optional
- ▶ Searchable thing attributes - optional
- ▶ Thing groups - optional
- ▶ Billing group - optional
- ▶ Packages and versions - optional

Device Shadow Info

Device Shadows allow connected devices to sync states with AWS. You can also get, update, or delete the state information of this thing's shadow using either HTTPs or MQTT topics.

☒ No shadow

☐ Named shadow

Create multiple shadows with different names to manage access to properties, and logically group

Figure 3 naming the thing

4- mark the first option and press next

Configure device certificate - *optional* [Info](#)

A device requires a certificate to connect to AWS IoT. You can choose how to register a certificate for your device now, or you can create and register a certificate for your device later. Your device won't be able to connect to AWS IoT until it has an active certificate with an appropriate policy.

Device certificate

☒ **Auto-generate a new certificate (recommended)**
Generate a certificate, public key, and private key using AWS IoT's certificate authority.

☐ **Use my certificate**
Use a certificate signed by your own certificate authority.

☐ **Upload CSR**
Register your CA and use your own certificates on one or many devices.

☐ **Skip creating a certificate at this time**
You can create a certificate for this thing and attach a policy to the certificate at a later time.

Cancel

Previous

Next

Figure 4 mark Auto-generate new certificate option

5- Then choose create policy option and paste this text

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "iot:Publish",
        "iot:Subscribe",
        "iot:Connect",
        "iot:Receive"
      ],
      "Resource": "*"
    }
  ]
}
```

6-finally, click on create thing

2. C++ Development Environment:

- Ensure you have a C++ compiler such as g++ installed.
- Make sure cmake is installed for building the AWS SDK.
- Install required libraries like libssl-dev and nlohmann/json.

3. AWS SDK for C++:

- Install the AWS SDK for C++ to interact with AWS services.

Install Dependencies

To install the required dependencies and AWS SDK for C++ [1], follow these steps:

- 1- Install System Dependencies:** Open the terminal and run the following commands to install cmake and OpenSSL development libraries:

```
sudo apt-get update
sudo apt-get install libcurl4-openssl-dev libssl-dev uuid-dev zlib1g-dev libpulse-dev
sudo apt install nlohmann-json3-dev
```

- 2- Clone and Build AWS SDK for C++:** Download and build the AWS SDK for C++:

```
# Clone the SDK repository
git clone --recurse-submodules https://github.com/aws/aws-sdk-cpp.git
cd aws-sdk-cpp

# Create a build directory
mkdir build && cd build

# Configure and build the SDK

cmake .. -DCMAKE_BUILD_TYPE=Debug -
DCMAKE_PREFIX_PATH=/usr/local/ -
DCMAKE_INSTALL_PREFIX=/usr/local/ -DBUILD_ONLY="iot"

make

# Install the SDK
sudo make install
```

Error while building the sdk

```
[ 89%] Linking CXX executable lot-gen-tests
s2n_init() failed: 402653268 (The libcrypto major version name seen at compile-time is different from the major version name seen at run-time)
Fatal error condition occurred in /home/zina/Desktop/cloud/aws-sdk-cpp/crt/aws-crt-cpp/crt/aws-c-io/source/s2n/s2n_tls_channel_handler.c:228: 0 && "s2n_init() failed"
Exiting Application
#####
Stack trace:
#####
/home/zina/Desktop/cloud/aws-sdk-cpp/build/crt/aws-crt-cpp/crt/aws-c-common/libaws-c-common.so.1(aws_backtrace_print+0x61) [0x735dd5de10e1]
/home/zina/Desktop/cloud/aws-sdk-cpp/build/crt/aws-crt-cpp/crt/aws-c-common/libaws-c-common.so.1(aws_fatal_assert+0x4d) [0x735dd5dc26ad]
/home/zina/Desktop/cloud/aws-sdk-cpp/build/crt/aws-crt-cpp/crt/aws-c-io/libaws-c-io.so.1.0.0(+0x2c9f9) [0x735dd58909f9]
/home/zina/Desktop/cloud/aws-sdk-cpp/build/crt/aws-crt-cpp/crt/aws-c-io/libaws-c-io.so.1.0.0(aws_io_library_init+0x48) [0x735dd5882b68]
/home/zina/Desktop/cloud/aws-sdk-cpp/build/crt/aws-crt-cpp/crt/aws-c-mqtt/libaws-c-mqtt.so.1.0.0(aws_mqtt_library_init+0x20) [0x735dd5c8c820]
/home/zina/Desktop/cloud/aws-sdk-cpp/build/crt/aws-crt-cpp/libaws-crt-cpp.so( ZN3Aws3Crt9ApiHandleC1EP13aws_allocator+0x40) [0x735dd66f5f70]
/home/zina/Desktop/cloud/aws-sdk-cpp/build/src/aws-cpp-sdk-core/libaws-cpp-sdk-core.so( ZN3Aws13InitializeCrtEv+0x99) [0x735dd6162c19]
/home/zina/Desktop/cloud/aws-sdk-cpp/build/src/aws-cpp-sdk-core/libaws-cpp-sdk-core.so( ZN3Aws7InitAPIERKNS_10SDKOptionsE+0x4b5) [0x735dd6160d05]
/home/zina/Desktop/cloud/aws-sdk-cpp/build/generated/tests/lot-gen-tests/lot-gen-tests(+0x7098) [0x625261dd2098]
/lib/x86_64-linux-gnu/libc.so.6(+0x29d90) [0x735dd5a29d90]
/lib/x86_64-linux-gnu/libc.so.6(__libc_start_main+0x80) [0x735dd5a29e40]
/home/zina/Desktop/cloud/aws-sdk-cpp/build/generated/tests/lot-gen-tests/lot-gen-tests(+0x7155) [0x625261dd2155]
Subprocess aborted
make[2]: *** [generated/tests/lot-gen-tests/CMakeFiles/lot-gen-tests.dir/build.make:135: generated/tests/lot-gen-tests/lot-gen-tests] Error 1
make[2]: *** Deleting file 'generated/tests/lot-gen-tests/lot-gen-tests'
make[1]: *** [CMakeFiles/Makefile2:981: generated/tests/lot-gen-tests/CMakeFiles/lot-gen-tests.dir/all] Error 2
make: *** [Makefile:136: all] Error 2
zina@zina-virtual-machine:~/Desktop/cloud/aws-sdk-cpp/build$
```

Figure 5 error content

This error indicates a mismatch between the OpenSSL (libcrypto) version at compile-time and runtime. Specifically.

This happens when the version of OpenSSL (libcrypto) used during the build of the AWS SDK is different from the version that is available when running the application.

Steps done to resolve this error

1- Uninstall OpenSSL:

- remove the current OpenSSL installation

```
sudo apt-get remove --purge openssl
```

- This will remove OpenSSL and its associated libraries. To ensure that all related dependencies are also removed, run:

```
sudo apt-get autoremove
```

2- Reinstall OpenSSL:

- First, download OpenSSL version 1.1.1w from openssl website

```
wget https://www.openssl.org/source/openssl-1.1.1w.tar.gz
```

- Extract the folder:

```
tar -zxf openssl-1.1.1w.tar.gz
cd openssl-1.1.1w
```

- Configure the installation, This step specify the directory where OpenSSL will be installed:

```
./config --prefix=/usr/local/ssl --openssldir=/usr/local/ssl shared zlib
```

- Compile and install:

```
make  
sudo make install
```

3- Configure the System to Use the New OpenSSL Version:

Now, tell the system to use the newly installed OpenSSL version:

- Update the shared library cache:

```
sudo ldconfig
```

- Add the new OpenSSL binary to the system's PATH by editing .bashrc:

```
sudo nano ~/.bashrc
```

- Add the following line at the end of the file:

```
export PATH="/usr/local/ssl/bin:$PATH"
```

- Save and exit, then reload the .bashrc file:

```
source ~/.bashrc
```

4- Verify the OpenSSL Installation:

- Check OpenSSL Version Installed: Verify the installed version of OpenSSL on your system using the following command:

```
openssl version
```

- You should see something like:

```
OpenSSL 1.1.1w  xx XXX 20xx
```

- Unfortunately, it didn't work so I tried to use an EC2 instance (VM) on the cloud to solve this issue.

Steps to build the SDK on Cloud machine

1- Navigate through EC2 service and click on launch instance.

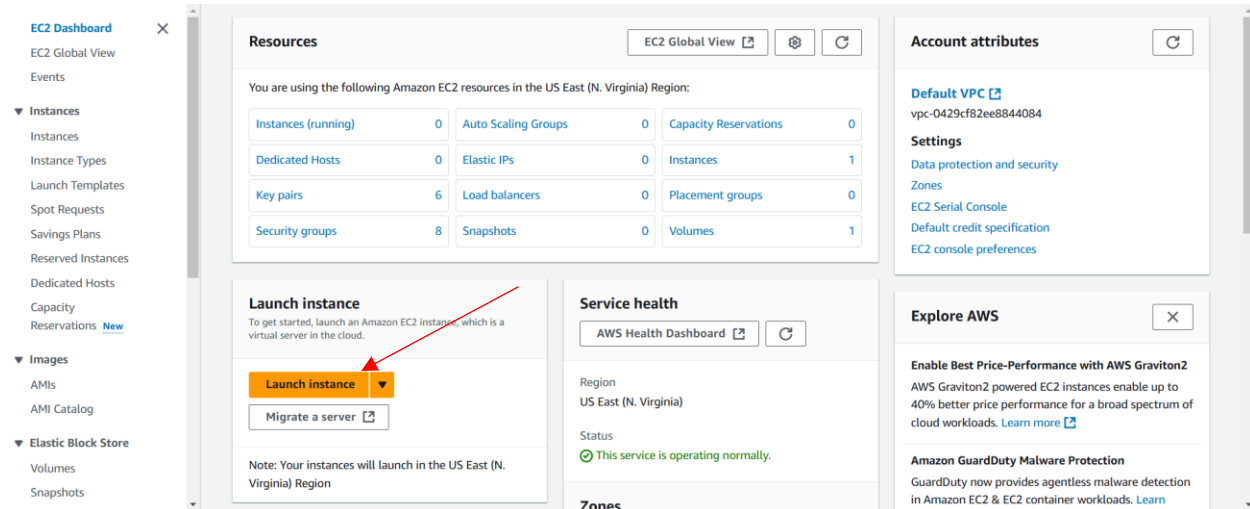


Figure 6 launch instance

2- Name the instance and choose the OS type

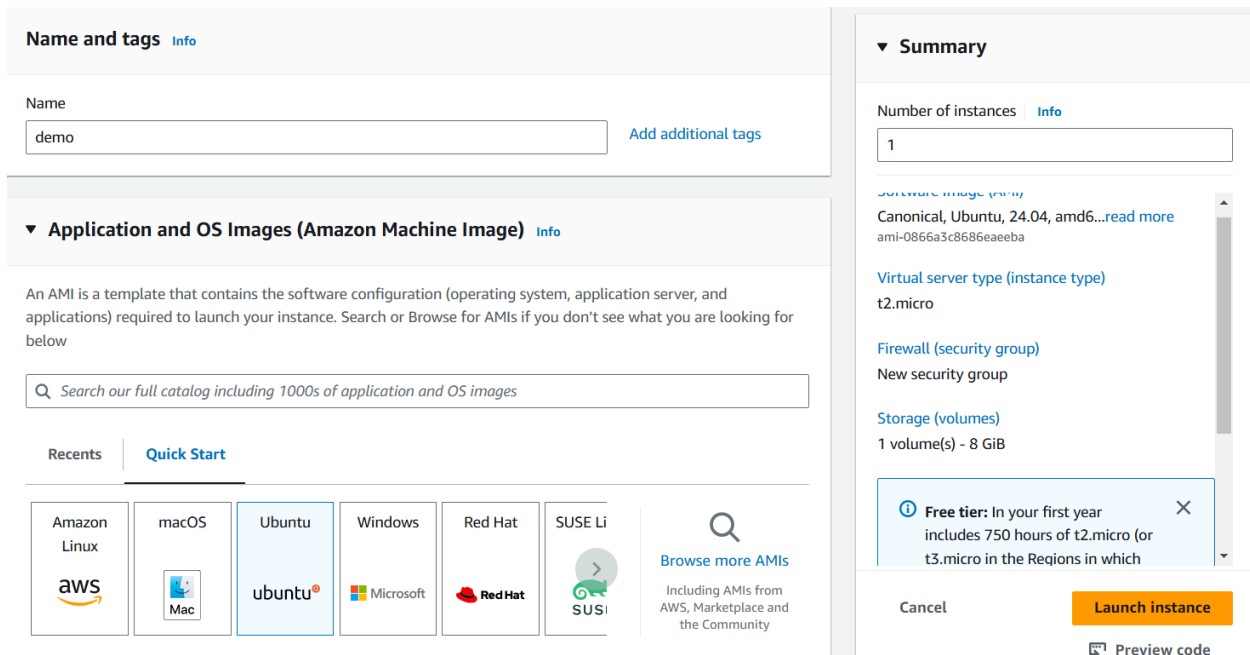
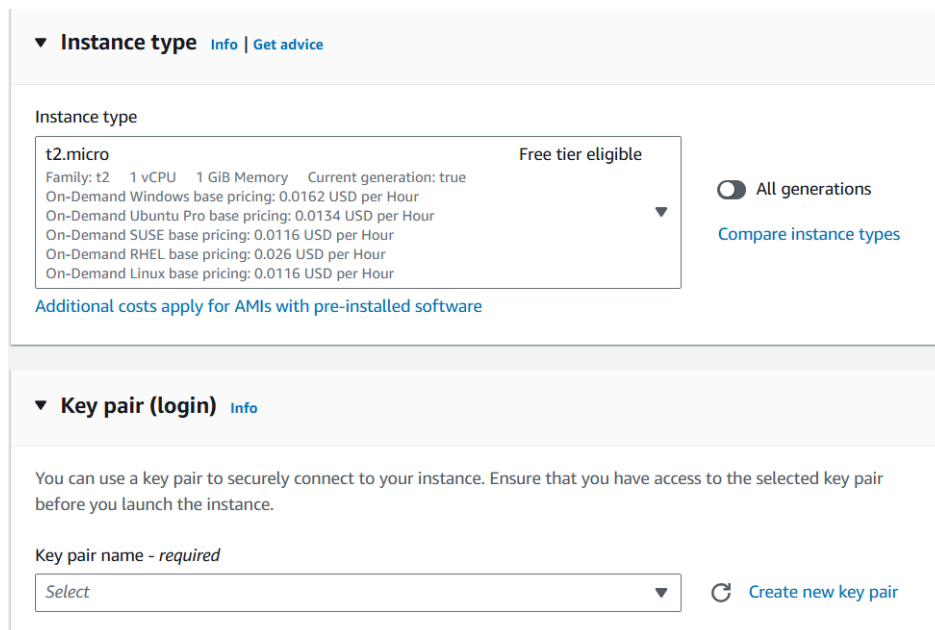


Figure 7 naming the instance and choosing OS type

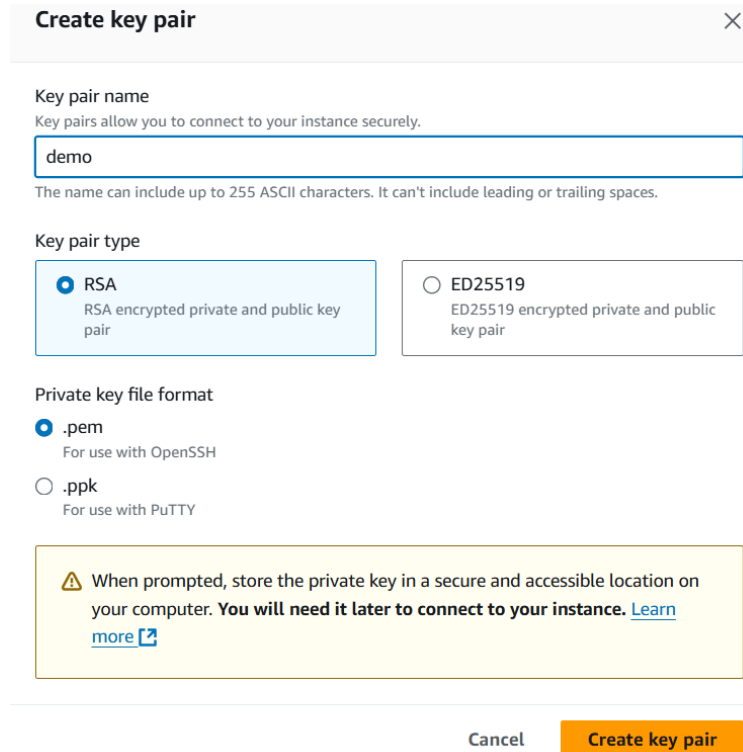
3- Choose the instance type (t2 micro as it is in the free tier), and then create a key pair



The screenshot shows the AWS console interface for creating an instance. The 'Instance type' section is expanded, showing the 't2.micro' instance type selected. It lists specifications: Family: t2, 1 vCPU, 1 GiB Memory, Current generation: true. Pricing information is provided for various operating systems: On-Demand Windows (0.0162 USD per Hour), On-Demand Ubuntu Pro (0.0134 USD per Hour), On-Demand SUSE (0.0116 USD per Hour), On-Demand RHEL (0.026 USD per Hour), and On-Demand Linux (0.0116 USD per Hour). A 'Free tier eligible' badge is present. To the right, there is a radio button for 'All generations' and a link to 'Compare instance types'. Below this, the 'Key pair (login)' section is expanded, showing a message about using a key pair to connect. A dropdown menu for 'Key pair name - required' is set to 'Select', and a 'Create new key pair' button is visible.

Figure 8 choose t2 micro as instance type

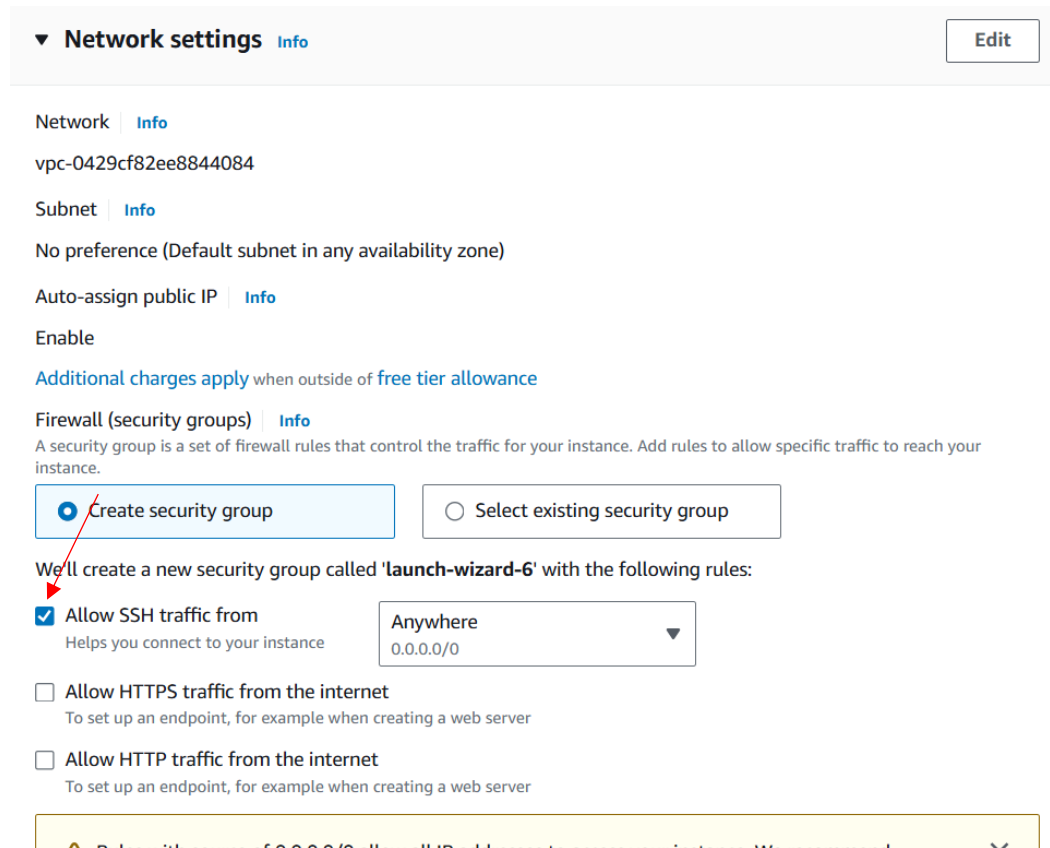
4- Name the key pair and save it in as we will use it to access the machine.



The screenshot shows the 'Create key pair' dialog box. The 'Key pair name' field is filled with 'demo'. Below it, a note states: 'The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.' Under 'Key pair type', the 'RSA' option is selected, with a description: 'RSA encrypted private and public key pair'. The 'ED25519' option is also available. Under 'Private key file format', the '.pem' option is selected, with a description: 'For use with OpenSSH'. The '.ppk' option is also available, with a description: 'For use with PuTTY'. A warning box at the bottom states: 'When prompted, store the private key in a secure and accessible location on your computer. You will need it later to connect to your instance. Learn more'. At the bottom right, there are 'Cancel' and 'Create key pair' buttons.

Figure 9 naming the key pair of the instance

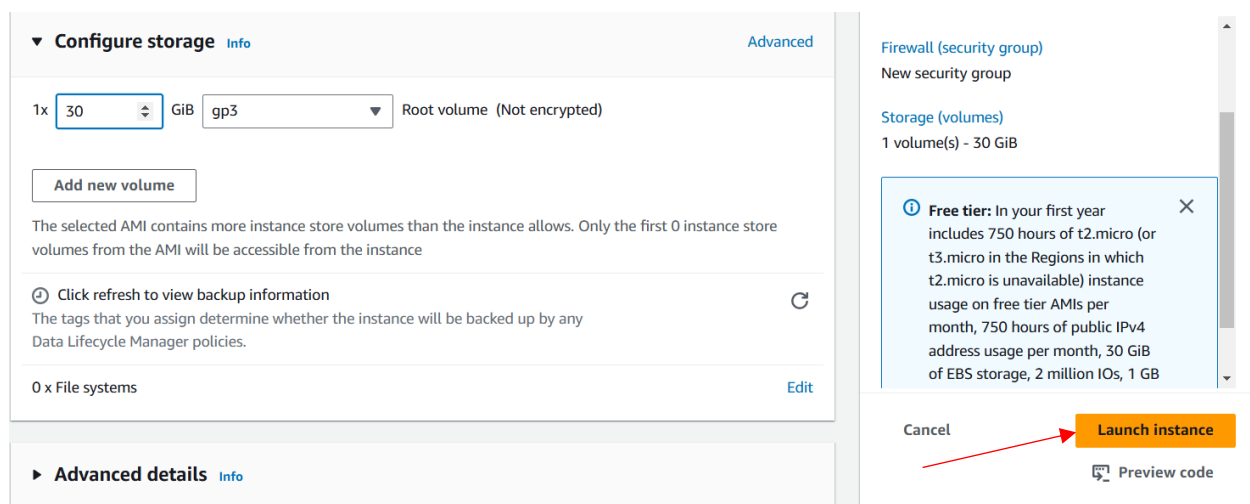
5- Make sure that “allow SSH traffic” is marked to make the machine can be accessed.



The screenshot shows the 'Network settings' section of the AWS EC2 instance configuration. It includes fields for 'Network' (vpc-0429cf82ee8844084), 'Subnet' (No preference), and 'Auto-assign public IP' (Enabled). The 'Firewall (security groups)' section is expanded, showing a radio button selection for 'Create security group' (selected) and 'Select existing security group'. Below this, a message states: 'We'll create a new security group called 'launch-wizard-6' with the following rules:'. A list of rules follows, with the first rule 'Allow SSH traffic from' checked. The source for this rule is set to 'Anywhere' (0.0.0.0/0). Other rules for HTTPS and HTTP traffic are unchecked. A yellow warning banner at the bottom states: 'Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend...'

Figure 10 marking "Allow SSH" option

6- Choose the storage (free tier provides up to 30 GB) and launch the instance



The screenshot shows the 'Configure storage' section of the AWS EC2 instance configuration. It displays '1x 30 GiB gp3' for the root volume. A message indicates that the selected AMI contains more instance store volumes than the instance allows. Below this, there is a 'Click refresh to view backup information' button. The 'Advanced details' section is also visible. On the right side, the 'Firewall (security group)' section shows 'New security group' and 'Storage (volumes)' shows '1 volume(s) - 30 GiB'. A 'Free tier' information box is displayed, stating: 'Free tier: In your first year includes 750 hours of t2.micro (or t3.micro in the Regions in which t2.micro is unavailable) instance usage on free tier AMIs per month, 750 hours of public IPv4 address usage per month, 30 GiB of EBS storage, 2 million IOs, 1 GB'. At the bottom right, there are 'Cancel' and 'Launch instance' buttons, with a red arrow pointing to the 'Launch instance' button. A 'Preview code' button is also present.

Figure 11 choosing suitable storage size for the instance

- After creation of the machine, I have tried to do the same steps provided by AWS documentation but I don't know why the machine stops always while building the SDK (make command).
- Here is a screenshot for this case:

4

```

[ 82%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/crypto/CRC32.cpp.o
[ 82%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/crypto/Cipher.cpp.o
[ 82%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/crypto/ContentCryptoMaterial.cpp.o
[ 83%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/crypto/ContentCryptoScheme.cpp.o
[ 83%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/crypto/CryptoBuf.cpp.o
[ 83%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/crypto/CryptoStream.cpp.o
[ 83%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/crypto/EncryptionMaterials.cpp.o
[ 83%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/crypto/KeyWrapAlgorithm.cpp.o
[ 83%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/crypto/MD5.cpp.o
[ 84%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/crypto/PrecalculatedHash.cpp.o
[ 84%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/crypto/Sha1.cpp.o
[ 84%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/crypto/Sha256.cpp.o
[ 84%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/crypto/Sha256HMAC.cpp.o
[ 84%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/crypto/factory/Factories.cpp.o
[ 84%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/event/EventDecoderStream.cpp.o
[ 85%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/event/EventEncoderStream.cpp.o
[ 85%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/event/EventHeader.cpp.o
[ 85%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/event/EventMessage.cpp.o
[ 85%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/event/EventStreamBuf.cpp.o
[ 85%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/event/EventStreamDecoder.cpp.o
[ 85%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/event/EventStreamEncoder.cpp.o
[ 85%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/event/EventStreamErrors.cpp.o
[ 86%] Building CXX object src/aws-cpp-sdk-core/CMakeFiles/aws-cpp-sdk-core.dir/source/utills/json/JsonSerializer.cpp.o
[ 86%] Linking CXX shared library libaws-cpp-sdk-core.so
[ 86%] Built target aws-cpp-sdk-core
[ 86%] Building CXX object generated/src/aws-cpp-sdk-iot/CMakeFiles/aws-cpp-sdk-iot.dir/ub1_IOT.cpp.o

```

i-03fbcc8ed22d21079
PublicIPs: 98.81.207.149 PrivateIPs: 172.31.37.90

Figure 12 Error while building the SDK on cloud machine

Using Python SDK

After encountering difficulties with the C++ SDK and the inability to resolve the issues effectively, I decided to explore the AWS IoT Core Python SDK. The Python SDK offers a wealth of resources and examples, making it easier to implement and test functionality compared to the C++ SDK.

Here are the detailed steps for that:

1- Click on “connect device”

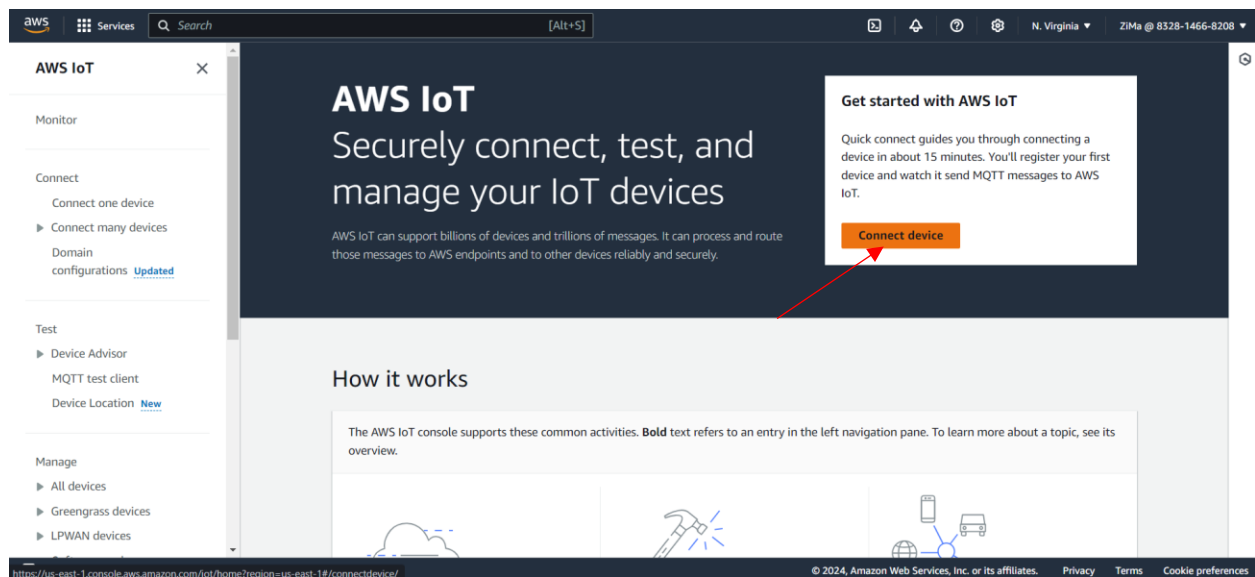


Figure 13 step 1 on downloading Python SDK

2- Write the name for the Thing and click on next.

The screenshot displays the 'Create a new thing' wizard in the AWS IoT console. At the top, there are two radio buttons: 'Create a new thing' (selected) and 'Choose an existing thing'. Below this is the 'Thing name' section, which includes a text input field containing 'Demo_Thing' and a note: 'Enter a unique name containing only: letters, numbers, hyphens, colons, or underscores. A thing name can't contain any spaces.' The 'Additional configurations' section lists several optional settings: 'Thing type', 'Searchable thing attributes', 'Thing groups', and 'Billing group'. At the bottom, a blue box contains an information icon and the text: 'Certificate and policy for your device. Your device requires a unique device certificate to securely authenticate its identity to AWS IoT, and an AWS IoT policy that authorizes it to send and receive messages. We'll create these resources for your device automatically. You can review and edit their properties later, if necessary.' At the very bottom are three buttons: 'Cancel', 'Previous', and 'Next'.

Figure 14 naming the thing

3- Choose OS and the language for the AWS SDK

Platform and SDK

Choose the platform OS and AWS IoT Device SDK that you want to use for your device.

Device platform operating system
This is the operating system installed on the device that will connect to AWS.

☒ **Linux / macOS**
Linux version: any
macOS version: 10.13+

☐ **Windows**
Version 10

AWS IoT Device SDK
Choose a Device SDK that's in a language your device supports.

☐ **Node.js**
Version 10+
Requires Node.js and npm to be installed

☒ **Python**
Version 3.6+
Requires Python and Git to be installed

☐ **Java**
Version 8
Requires Java JDK, Maven, and Git to be installed

Cancel Previous Next

Figure 15 Choosing OS of the machine and python

4- Download the connection kit and copy this command to unzip it.

Connection kit

Certificate Demo_Thing.cert.pem	Private key Demo_Thing.private.key	AWS IoT Device SDK Python
Script to send and receive messages start.sh	Policy Demo_Thing-Policy View policy	

Download

If you are running this from a browser on the device, after you download the connection kit, it will be in the browser's download folder.

If you are not running this from a browser on your device, you'll need to transfer the connection kit from your browser's download folder to your device using the method you tested when you prepared your device in step 1.

Download connection kit

Unzip connection kit on your device

After the connection kit is on your device, unzip it using this command:

```
unzip connect_device_package.zip
```

Copy

Figure 16 downloading the connection kit

- 5- Finally, type these commands on the terminal (first one to make the bash script executable and second one to run it), and we should see messages below “hello message” from the machine after running the script.

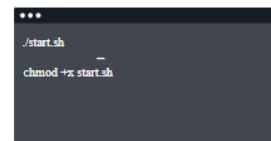
How to display messages from your device

Step 1: Add execution permissions

On the device, launch a terminal window to copy and paste the command to add execution permissions.

```
chmod +x start.sh
```

 Copy



Step 2: Run the start script

On the device, copy and paste the command to the terminal window and run the start script.

```
./start.sh
```

 Copy



Step 3: Return to this screen to view your device's messages

After running the start script, return to this screen to see the messages between your device and AWS IoT. The messages from your device appear in the following list.

Subscriptions sdk/test/python Pause Clear

sdk/test/python

Waiting for messages

Figure 17 final steps to send messages from the machine using python sdk

Creating DB to Hold incoming data

Now we have ensured that AWS IOT CORE service works properly, we need to attach a role to it so we can get the messages in a database for better observation and analysis.

Here are the detailed steps for this task:

- 1- navigate through amazon timestream database and click on create database

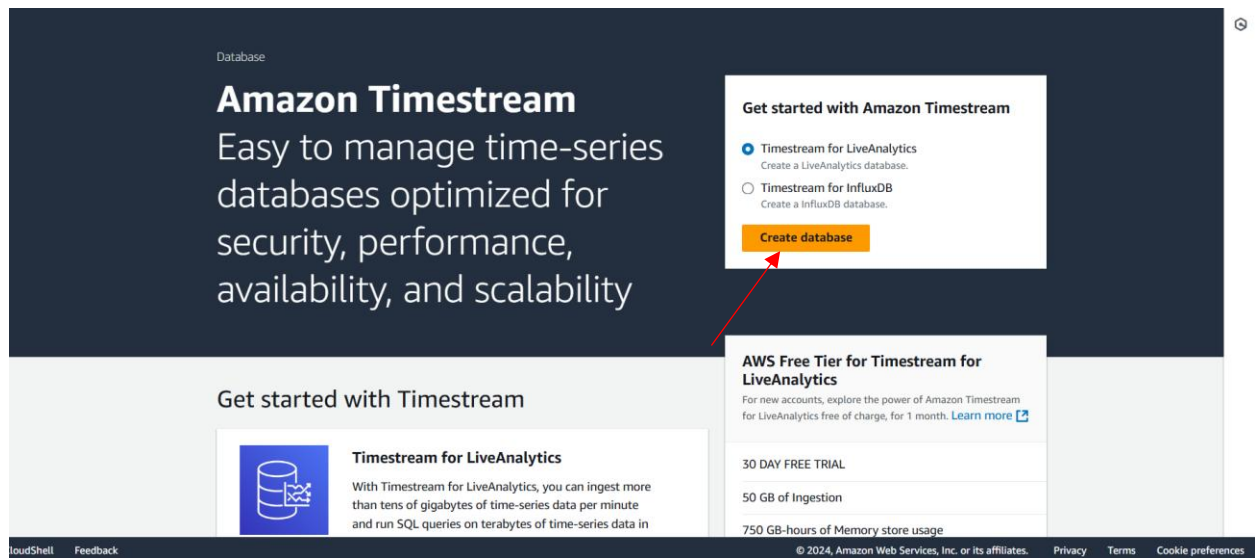


Figure 18 step 1 on creating the DB

- 2- Name the database

Figure 19 naming the DB

3- Keep the default encryption option as it is

Encryption
All Amazon Timestream data is encrypted by default.

KMS key
KMS key IDs and aliases appear in the list after they have been created using the Key Management Service (KMS) console.

aws/timestream ▼
Manage keys ↗

Description
Default key that protects my Timestream data when no other key is defined

Key ARN
 arn:aws:kms:us-east-1:832814668208:key/3c7e6145-4d81-4b1c-8a53-7efe9a5feedd

Figure 20 encryption option

4- press on create database

- After that we need to create a table to hold the incoming data

Creating a table for the DB

1- Click on create table

TEST_THINGDB Edit Delete

Summary

Name TEST_THINGDB	Modified time (UTC) 10/25/2024, 4:24:20 PM	Creation time (UTC) 10/25/2024, 4:24:20 PM
Database ARN  arn:aws:timestream:us-east-1:832814668208:database/TEST_THINGDB	KMS key ARN arn:aws:kms:us-east-1:832814668208:key/3c7e6145-4d81-4b1c-8a53-7efe9a5feedd	

Monitoring **Tables** Tags

Tables (0) Info ↻ Create backup Create scheduled query Edit Delete **Create table**

< 1 > ⚙

Table name ▼	Creation time (UTC) ▼
 	

Figure 21 Step 1 on creating a table

2- Name it.

3- Specify the memory & magnetic store retention.

Data retention [Info](#)

Specify how long your data is retained in each storage tier. Data moves from the memory store to the magnetic store as it ages. Data that exceeds the magnetic store retention will be deleted.

Memory store retention

Specify how long data will be stored in the memory store before it is moved to magnetic store.

The value must be a number. Minimum 1 hour, maximum 12 months.

Magnetic store retention

Specify how long data will be stored in magnetic store before it is deleted. If this value is less than the original magnetic restore retention period, data will be lost.

The value must be a number. Minimum 1 day, maximum 200 years.

Magnetic Storage Writes

Settings

☐ Enable magnetic storage writes

Figure 22 the memory & magnetic store retention.

4- Click on create table.

- Then we need to make a rule for the iot thing to send the data to this table.

Creating a rule for the IoT thing to send data to the created table

1- Navigate through iot core service and click on create role.

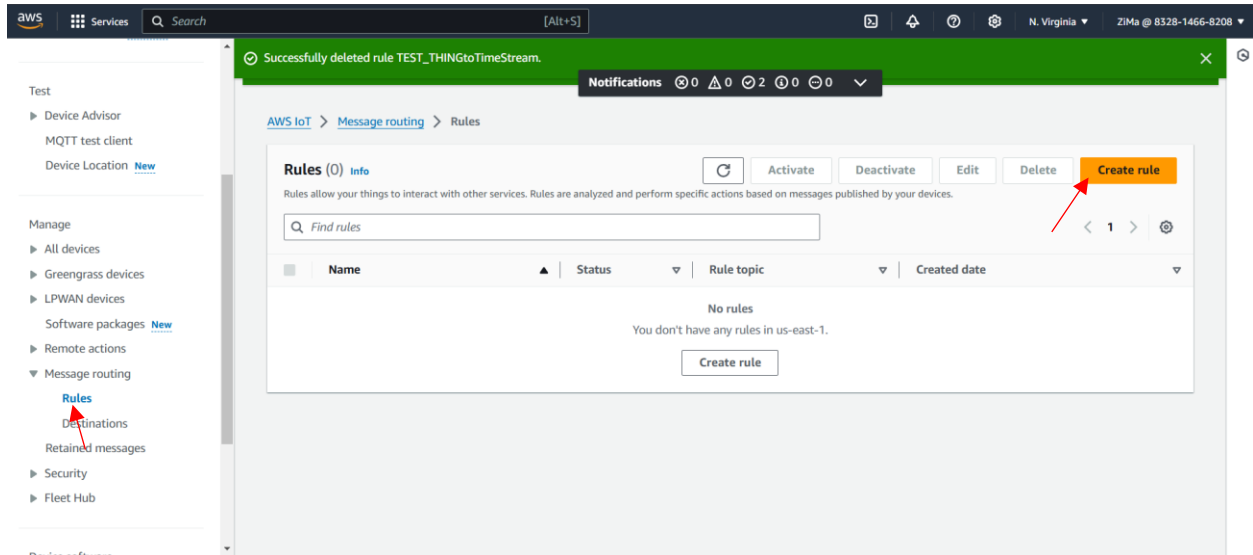


Figure 23 creating a rule

2- Name the role and click on next.

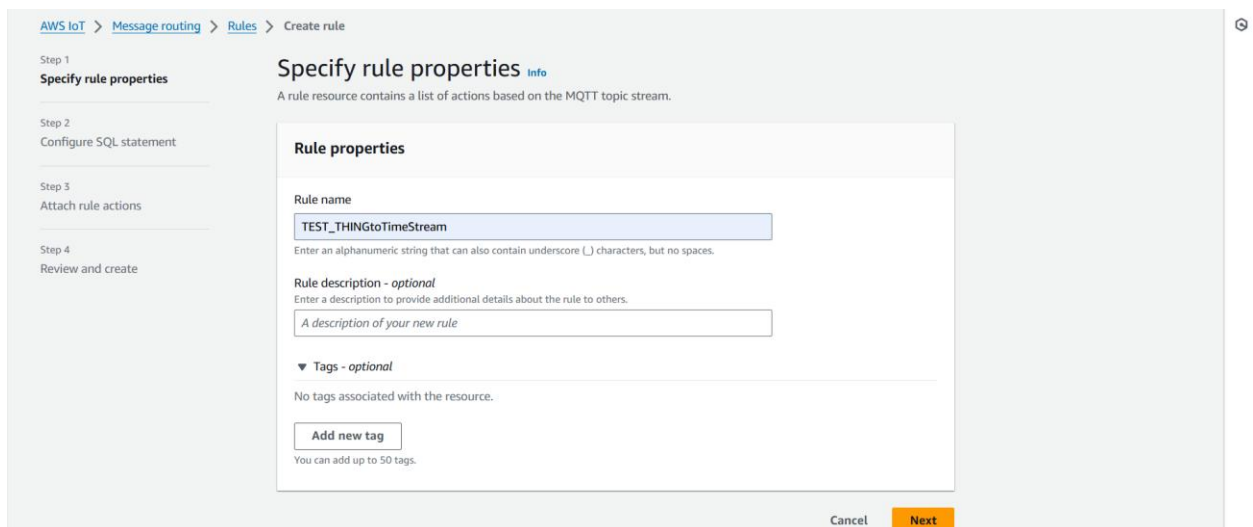


Figure 24 naming the rule

- 3- Enter this SQL statement to get all the messages from the desired topic.

Configure SQL statement [Info](#)

Add a simplified SQL syntax to filter messages received on an MQTT topic and push the data elsewhere.

SQL statement

SQL version
The version of the SQL rules engine to use when evaluating the rule.

2016-03-23 ▼

SQL statement
Enter a SQL statement using the following: `SELECT <Attribute> FROM <Topic Filter> WHERE <Condition>`. For example: `SELECT temperature FROM 'iot/topic' WHERE temperature > 50`. To learn more, see [AWS IoT SQL Reference](#).

1 `SELECT * FROM 'sdk/test/python'`

Figure 25 configure SQL statement

- 4- Choose Timestream table in the action & database name & table name & choose host and quality in the dimension name and put their values from the code then also specify the time stamp also from the code.

Timestream table [Info](#) Remove

Write a message into a Timestream table

Database name [Info](#)

TEST_THINGDB Copy View

Create Timestream database

Table name

TestThingTable Copy View

Create Timestream table

Dimensions
Each record contains an array of dimensions (minimum 1). Dimensions represent the metadata attributes of a time series data point.

Dimensions name	Dimension value	
host	\${hostname}	Remove
quality	\${quality}	Remove

Add new dimension

Timestamp value - optional Timestamp unit

\${time} SECONDS ▼

Figure 26 table parameters

5- Press next then create

- Now we will apply Query test to ensure that it functions in a right way.

- Write this in the editor:

```
SELECT
time,
host,
quality,
MAX(CASE WHEN measure_name = 'memory_percent' THEN measure_value::double END) AS memory_percent,
MAX(CASE WHEN measure_name = 'disk_usage_percent' THEN measure_value::double END) AS disk_usage_percent,
MAX(CASE WHEN measure_name = 'cpu_usage_percent' THEN measure_value::double END) AS cpu_usage_percent
FROM "TEST_THINGDB"."TestThingTable"
GROUP BY time, host, quality
ORDER BY time DESC
```

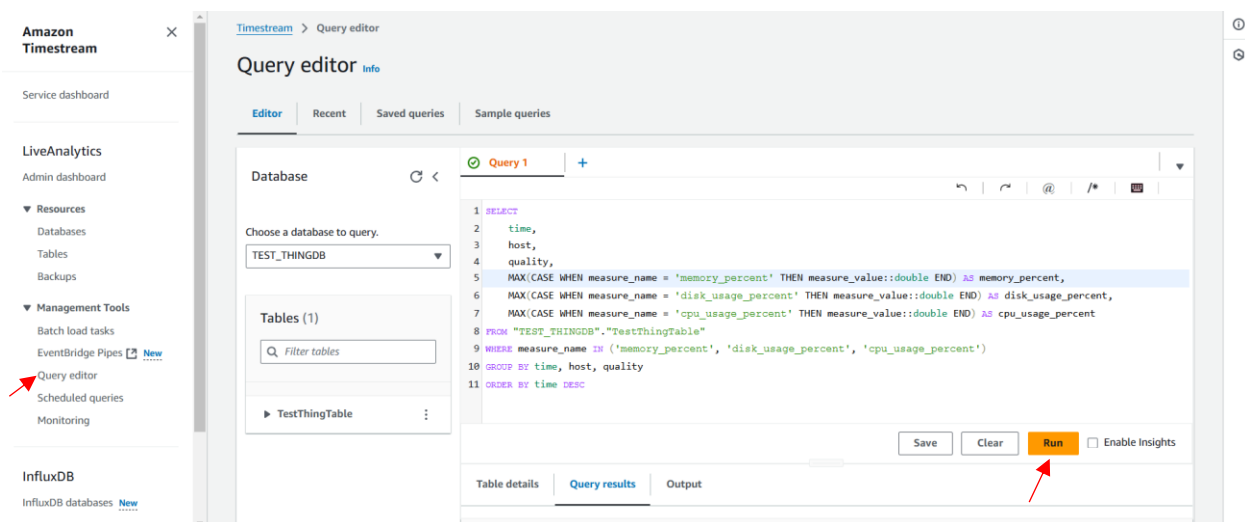


Figure 27 Query test

- The query result:

Table details Query results Output					
Rows returned (3)					
< 1 > ⚙					
time	host	quality	memory_percent	disk_usage_percent	cpu_usage_percent
2024-10-25 20:00:32.000000000	labtob	good	25.6	53.6	0.9
2024-10-25 20:00:30.000000000	labtob	good	25.5	53.6	0.5
2024-10-25 20:00:28.000000000	labtob	good	25.5	53.6	1.8

Figure 28 Query test results

Another trial for building C++ SDK

- I have made another trial to do this task with C++ but now with another SDK version that worked properly in my machine, here are the done steps:

1- install prerequisite packages

```
sudo apt update
sudo apt install -y cmake git libssl-dev uuid-dev
```

2- Clone the AWS IoT SDK for C++

```
git clone --recurse-submodules https://github.com/aws/aws-iot-device-sdk-cpp-v2.git
```

3- Build the SDK

```
cd aws-iot-device-sdk-cpp-v2
mkdir build && cd build
cmake .. -DBUILD_SHARED_LIBS=ON
make
sudo make install
```

- 4- Create a file with name “publish_to_aws.cpp” that contains the code to publish and subscribe messages to the IoT core service [2].

You can find the code for publishing and receiving messages in this link:

https://drive.google.com/file/d/1_7HTYEIIyB4oasBcFkUt5aC3n5abnNSW/view?usp=drive_link

- 5- Run the code with these commands

```
g++ publish_to_aws.cpp -o publish_to_aws -laws-crt-cpp -laws-c-common  
./publish_to_aws
```

- 6- monitor the messages in the service using MQTT test client option in the service

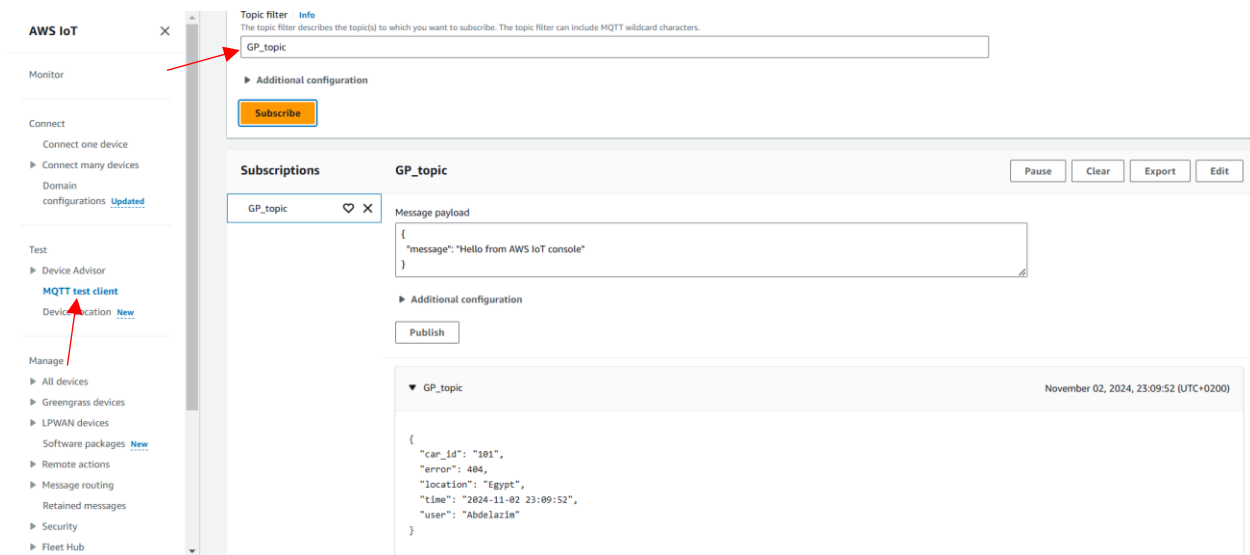


Figure 29 MQTT test client

- 7- and then create the database and the table as the previous steps that explained in the python SDK section

8- Now we will apply Query test to ensure that it functions in a right way.

- Write this in the editor:

```
SELECT
time,
MAX(CASE WHEN measure_name = 'car_id' THEN measure_value::varchar END) AS car_id,
MAX(CASE WHEN measure_name = 'user' THEN measure_value::varchar END) AS user,
MAX(CASE WHEN measure_name = 'location' THEN measure_value::varchar END) AS location,
MAX(CASE WHEN measure_name = 'error' THEN measure_value::bigint END) AS error_code
FROM "TestThingDB"."TestThingTable"
GROUP BY time
ORDER BY time DESC
```

The screenshot shows a query editor interface. On the left, there's a sidebar with 'Database' and 'Tables (1)' sections. The 'Database' section shows 'TestThingDB' selected. The 'Tables (1)' section shows 'TestThingTable' selected. The main area displays a SQL query in a text editor. Below the editor, there are buttons for 'Save', 'Clear', 'Run', and 'Enable insights'. The 'Query results' tab is active, showing 'Rows returned (3)'. The results are displayed in a table with 5 columns: time, car_id, user, location, and error_code. The table contains 3 rows of data.

time	car_id	user	location	error_code
2025-02-17 09:27:34.207000000	101	Abdelazim	Egypt	404
2025-02-17 09:27:29.206000000	101	Abdelazim	Egypt	404
2025-02-17 09:20:12.174000000	101	Abdelazim	Egypt	404

Figure 30 Query results

References

[1] <https://docs.aws.amazon.com/sdk-for-cpp/v1/developer-guide/setup-linux.html>

[2] <https://github.com/aws/aws-iot-device-sdk-cpp-v2/tree/main/samples>